

Protocols and Datatypes

Abstractions lay at the foundation of reusable code. The Clojure language itself has abstractions for sequences, collections, and callability. Traditionally, these abstractions were described with Java interfaces and implemented using Java classes. In the beginning, Clojure provided proxy and genclass, removing the need to drop all the way to Java to achieve this, but that has changed with the introduction of protocols.

- Protocols provide an alternative to Java interfaces for high-performance polymorphic method dispatch.
- Datatypes provide an alternative to Java classes for creating implementations of abstractions defined with either protocols or interfaces.

Protocols and datatypes provide a high-performance, flexible mechanism for abstraction and concretion that removes the need to write Java interfaces and classes when programming in Clojure. With protocols and datatypes, you can create new abstractions and new types that implement those abstractions and even extend new abstractions to existing types.

In this chapter, we will explore Clojure's approach to abstraction using protocols and datatypes. First, we will implement our own version of Clojure's built-in spit and slurp functions. Then, we will take a short detour to build a CryptoVault, where you will learn about extending some of Java's standard library. Finally, we will put everything together using records and protocols to define musical notes and sequences. After working through these exercises, you will certainly see the power of Clojure's composable abstractions.

6.1 Programming to Abstractions

Clojure's spit and slurp I/O functions are built on two abstractions, reading and writing. This means you can use them with a variety of source and desti-

nation types, including files, URLs, and sockets, and they can be extended to support new types by anybody, whether they are existing types or newly defined.

- The slurp function takes an input source, reads the contents, and returns it as a string.
- The spit function takes an output destination and a value, converts the value to a string, and writes it to the output destination.

We will start by writing basic versions of the two functions that can read from and write to files only. We will then refactor the basic versions several times as we explore different approaches to supporting additional datatypes. Working through this will give you a good feel for the usefulness of programming to abstractions in general and the flexibility and power of Clojure's protocols and datatypes in particular.

After writing our versions of spit and slurp, called expectorate and gulp, respectively, that work with several existing datatypes, we will create a new datatype, CryptoVault, that can be used with our versions of the functions as well as the originals.

gulp and expectorate

The gulp function is a simplified version of Clojure's slurp function, and expectorate, despite its highfalutin name, is a dumbed-down version of Clojure's spit function. Let's write a basic version of gulp that can read from a java.io.File only.

```
src/examples/gulp.clj
(ns examples.gulp
  (:import (java.io FileInputStream InputStreamReader BufferedReader)))
(defn gulp [src]
  (let [sb (StringBuilder.)]
    (with-open [reader (-> src
                             FileInputStream.
                             InputStreamReader.
                             BufferedReader.)]
      (loop [c (.read reader)]
        (if (neg? c)
          (str sb)
          (do
             (.append sb (char c))
             (recur (.read reader))))))))
```

The gulp function creates a BufferedReader from a given File object and then loops/recurs over it, reading a character at a time and appending each to a StringBuilder until it reaches the end of the input where it returns a string. The basic expectorate function is even smaller:

```
src/examples/expectorate.clj
```

```
(ns examples.expectorate
  (:import (java.io FileOutputStream OutputStreamWriter BufferedWriter)))

(defn expectorate [dst content]
  (with-open [writer (-> dst
                          FileOutputStream.
                          OutputStreamWriter.
                          BufferedWriter.)]
    (.write writer (str content))))
```

It creates a `BufferedWriter` file, converts the value of the `content` parameter to a string, and writes it out to the `BufferedWriter`.

But what if we want to support additional types like `Sockets`, `URLs`, and basic input and output streams? We need to update `gulp` and `expectorate` to be able to make `BufferedReaders` and `BufferedWriters` from datatypes other than files. So, let's create two new functions, `make-reader` and `make-writer`, that will be responsible for this behavior.

- The `make-reader` function makes a `BufferedReader` from an input source.
- The `make-writer` makes a `BufferedWriter` from an output destination.

```
(defn make-reader [src]
  (-> src FileInputStream. InputStreamReader. BufferedReader.))

(defn make-writer [dst]
  (-> dst FileOutputStream. OutputStreamWriter. BufferedWriter.))
```

Like our basic `gulp` and `expectorate` functions, `make-reader` and `make-writer` work only on files, but that will change shortly. Now let's refactor `gulp` and `expectorate` to use the new functions:

```
src/examples/protocols.clj
```

```
(defn gulp [src]
  (let [sb (StringBuilder.)]
    (with-open [reader (make-reader src)]
      (loop [c (.read reader)]
        (if (neg? c)
          (str sb)
          (do
             (.append sb (char c))
             (recur (.read reader))))))))

(defn expectorate [dst content]
  (with-open [writer (make-writer dst)]
    (.write writer (str content))))
```

We can now add support for additional source and destination types to `gulp` and `expectorate` just by updating `make-reader` and `make-writer`. One approach to supporting additional types is to use a `cond`, or `condp`, statement to process different types appropriately. For example, the following version of `make-reader` replaces the call to the `FileInputStream` constructor with a `condp` statement that creates an `InputStream` from the given input, whether it is a `File`, `Socket`, or `URL` or already is an `InputStream`.

```
(defn make-reader [src]
  (-> (condp = (type src)
    java.io.InputStream src
    java.lang.String (FileInputStream. src)
    java.io.File (FileInputStream. src)
    java.net.Socket (.getInputStream src)
    java.net.URL (if (= "file" (.getProtocol src))
      (-> src .getPath FileInputStream.)
      (.openStream src)))
    InputStreamReader.
    BufferedReader.))
```

Here's a version of `make-writer` using the same strategy:

```
(defn make-writer [dst]
  (-> (condp = (type dst)
    java.io.OutputStream dst
    java.io.File (FileOutputStream. dst)
    java.lang.String (FileOutputStream. dst)
    java.net.Socket (.getOutputStream dst)
    java.net.URL (if (= "file" (.getProtocol dst))
      (-> dst .getPath FileOutputStream.)
      (throw (IllegalArgumentException.
        "Can't write to non-file URL"))))
    OutputStreamWriter.
    BufferedWriter.))
```

The problem with this approach is that it is closed: nobody else can come along and add support for new source and destination types without rewriting `make-reader` and `make-writer`. What we need is an open solution, one where support for new types can be added after the fact and by different parties. What we need is two abstractions, one for reading and one for writing.

6.2 Interfaces

In Java, the usual mechanism for supporting this form of abstraction is the interface. The interface mechanism provides a means for dispatching calls to an abstract function, specified in an interface definition, to a specific implementation based on the datatype of the first parameter passed in the call. In

Java, the first parameter is implicit; it is the object that implements the interface.

The following are the strengths of interfaces:

- Datatypes can implement multiple interfaces.
- Interfaces provide only specification, not implementation, which allows implementation of multiple interfaces without the problems associated with multiple class inheritance.

The weakness of interfaces is that existing datatypes cannot be extended to implement new interfaces without rewriting them.

We can create Java interfaces in Clojure with the `definterface` macro. This takes a name and one or more method signatures:

```
(definterface name & sigs)
```

Let's create our abstraction for things-that-can-read-from-and-be-written-to as an interface, which we'll call `IOFactory`.

```
(definterface IOFactory
  (^java.io.BufferedReader make-reader [this])
  (^java.io.BufferedWriter make-writer [this]))
```

This will create an interface called `IOFactory` that includes two abstract functions, `make-reader` and `make-writer`. Any class that implements this interface must include `make-reader` and `make-writer` functions that take a single parameter and an instance of the datatype itself and that return a `BufferedReader` and `BufferedWriter`, respectively.

Unfortunately, the interfaces that a class supports are determined at design time by the author; once a Java class is defined, it cannot be updated to support new interfaces without rewriting it. Therefore, we can't extend the `File`, `Socket`, and `URL` classes to implement the `IOFactory` interface.

Like the versions of `make-reader` and `make-writer` we based on `condp`, our interface is closed to extension by parties other than the author. This is part of what is called the *expression problem*.¹ Fortunately, Clojure has a solution to it.²

6.3 Protocols

One piece of Clojure's solution to the expression problem is the protocol. Protocols provide a flexible mechanism for abstraction that leverages the best

1. <http://lambda-the-ultimate.org/node/2232>

2. <http://www.ibm.com/developerworks/java/library/j-clojure-protocols/?ca=drs->

parts of interfaces by providing only specification, not implementation, and by letting datatypes implement multiple protocols. Additionally, protocols address the key weaknesses of interfaces by allowing nonintrusive extension of existing types to support new protocols.

The following are the strengths of protocols:

- Datatypes can implement multiple protocols.
- Protocols provide only specification, not implementation, which allows implementation of multiple interfaces without the problems associated with multiple-class inheritance.
- Existing datatypes can be extended to implement new interfaces with no modification to the datatypes.
- Protocol method names are namespaced, so there is no risk of name collision when multiple parties choose to extend the same extant type.

The `defprotocol` macro works just like `definterface`, but now we are able to extend existing datatypes to implement our new abstraction.

```
(defprotocol name & opts+sigs)
```

Let's redefine `IOFactory` as a protocol, instead of an interface.

```
(defprotocol IOFactory
  "A protocol for things that can be read from and written to."
  (make-reader [this] "Creates a BufferedReader.")
  (make-writer [this] "Creates a BufferedWriter."))
```

Notice we can include a document string for the protocol as a whole, as well as for each of its methods. Now let's extend `java.io.InputStream` and `java.io.OutputStream` to implement our `IOFactory` protocol.

We use the `extend` function to associate an existing type to a protocol and to provide the required function implementations, usually referred to as *methods* in this context. The parameters to `extend` are the name of the type to extend, the name of the protocol to implement, and a map of method implementations, where the keys are keywordized versions of the method names.

```
(extend type & proto+mmaps)
```

The `make-reader` implementation for an `InputStream` just wraps the value passed to it in a `BufferedReader`.

```
src/examples/protocols.clj
(extend InputStream
  IOFactory
  {:make-reader (fn [src]
```

```

        (-> src InputStreamReader. BufferedReader.))
:make-writer (fn [dst]
  (throw (IllegalArgumentException.
    "Can't open as an InputStream.))))))

```

Similarly, the implementation of `make-writer` for an `OutputStream` wraps its given input in a `BufferedWriter`. And since you can't write to an `InputStream` or read from an `OutputStream`, the respective implementations of `make-writer` and `make-reader` throw `IllegalArgumentException`s.

```

(extend OutputStream
  IOFactory
  {:make-reader (fn [src]
    (throw (IllegalArgumentException.
      "Can't open as an OutputStream.)))
  :make-writer (fn [dst]
    (-> dst OutputStreamWriter. BufferedWriter.))})

```

We can extend the `java.io.File` type to implement our `IOFactory` protocol with the `extend-type` macro, which provides a slightly cleaner syntax than `extend`.

```
(extend-type type & specs)
```

It takes the name of the type to extend and one or more specs, which includes a protocol name and its respective method implementations.

```

(extend-type File
  IOFactory
  (make-reader [src]
    (make-reader (FileInputStream. src)))
  (make-writer [dst]
    (make-writer (FileOutputStream. dst))))

```

Notice that we create an `InputStream`, specifically, a `FileInputStream`, from our file and then make a recursive call to `make-reader`, which will be dispatched to the implementation defined earlier for `InputStream`s. We use the same recursive pattern for the `make-writer` method, as well as for the methods of the following remaining types.

We can extend the remaining types all at once with the `extend-protocol` macro:

```
(extend-protocol protocol & specs)
```

This takes the name of the protocol followed by one or more type names with their respective method implementations.

```

(extend-protocol IOFactory
  Socket
  (make-reader [src]
    (make-reader (.getInputStream src))))

```

```

(make-writer [dst]
  (make-writer (.getOutputStream dst)))

URL
(make-reader [src]
  (make-reader
    (if (= "file" (.getProtocol src))
      (-> src .getPath FileInputStream.)
      (.openStream src))))

(make-writer [dst]
  (make-writer
    (if (= "file" (.getProtocol dst))
      (-> dst .getPath FileInputStream.)
      (throw (IllegalArgumentException.
        "Can't write to non-file URL"))))))

```

Now let's put it all together.

```

(ns examples.io
  (:import (java.io File FileInputStream FileOutputStream
    InputStream InputStreamReader
    OutputStream OutputStreamWriter
    BufferedReader BufferedWriter)
    (java.net Socket URL)))

(defprotocol IOFactory
  "A protocol for things that can be read from and written to."
  (make-reader [this] "Creates a BufferedReader.")
  (make-writer [this] "Creates a BufferedWriter."))

(defn gulp [src]
  (let [sb (StringBuilder.)]
    (with-open [reader (make-reader src)]
      (loop [c (.read reader)]
        (if (neg? c)
          (str sb)
          (do
            (.append sb (char c))
            (recur (.read reader)))))))

(defn expectorate [dst content]
  (with-open [writer (make-writer dst)]
    (.write writer (str content))))

(extend-protocol IOFactory
  InputStream
  (make-reader [src]
    (-> src InputStreamReader. BufferedReader.))

```

```

(make-writer [dst]
  (throw
    (IllegalArgumentException.
      "Can't open as an InputStream.)))

OutputStream
(make-reader [src]
  (throw
    (IllegalArgumentException.
      "Can't open as an OutputStream.)))

(make-writer [dst]
  (-> dst OutputStreamWriter. BufferedWriter.))

File
(make-reader [src]
  (make-reader (FileInputStream. src)))

(make-writer [dst]
  (make-writer (FileOutputStream. dst)))

Socket
(make-reader [src]
  (make-reader (.getInputStream src)))

(make-writer [dst]
  (make-writer (.getOutputStream dst)))

URL
(make-reader [src]
  (make-reader
    (if (= "file" (.getProtocol src))
      (-> src .getPath FileInputStream.)
      (.openStream src))))

(make-writer [dst]
  (make-writer
    (if (= "file" (.getProtocol dst))
      (-> dst .getPath FileInputStream.)
      (throw (IllegalArgumentException.
        "Can't write to non-file URL"))))))

```

6.4 Datatypes

We have shown how to extend existing types to implement new abstractions with protocols, but what if we want to create a new type in Clojure? That is where datatypes come in.

A datatype provides the following:

- A unique class, either named or anonymous
- Structure, either explicitly as fields or implicitly as a closure
- Fields that can have type hints and can be primitive
- Optional implementations of abstract methods specified in protocols or interfaces
- Immutability on by default
- Unification with maps (via records)

We will use the `deftype` macro to define a new datatype, called `CryptoVault`, that will implement two protocols, including `IOFactory`.

Now that `gulp` and `expectorate` support several existing Java classes, let's create a new supported type, `CryptoVault`. You'll create an instance of a `CryptoVault` by passing in an argument that implements the `clojure.java.io.IOFactory` protocol (not the one we've defined here), a path to a cryptographic key store, and a password. The contents expectorated into the `CryptoVault` will be encrypted and written to the `IOFactory` object and then decrypted when gulped back in.

We'll use `deftype` to create the new type.

```
(deftype name [& fields] & opts+specs)
```

It takes the name of the type and a vector of fields contained by the type. The naming convention for datatypes is the same as used by Java classes, i.e., `PascalCase` or `CamelCase`.

```
user=> (deftype CryptoVault [filename keystore password])
user.CryptoVault
```

Once the type has been defined, we can create an instance of our `CryptoVault` like so:

```
user=> (def vault (->CryptoVault "vault-file" "keystore" "toomanysecrets"))
#'user/vault
```

And its fields can be accessed using the same prefix-dot syntax used to access fields in Java objects.

```
user=> (.filename vault)
"vault-file"
```

```
user=> (.keystore vault)
"keystore"
```

```
user=> (.password vault)
"toomanysecrets"
```

Now that we have defined the basic `CryptoVault` type, let's add behavior with some methods. Datatypes can implement only those methods that have been specified in either a protocol or an interface, so let's first create a Vault protocol.

```
(defprotocol Vault
  (init-vault [vault])
  (vault-output-stream [vault])
  (vault-input-stream [vault]))
```

The protocol includes three functions—`init-vault`, `vault-output-stream`, and `vault-input-stream`—that every Vault must implement.

We can define our new type's methods inline with `deftype`; we just pass the type name and vector of fields as before, followed by a protocol name and one or more method bodies:

```
src/examples/cryptovault.clj
(ns examples.cryptovault
  (:use [examples.io :only [IOFactory make-reader make-writer]])
  (:require [clojure.java.io :as io])
  (:import (java.security KeyStore KeyStore$SecretKeyEntry
                           KeyStore$PasswordProtection)
           (javax.crypto KeyGenerator Cipher CipherOutputStream
                           CipherInputStream)
           (java.io FileOutputStream)))
(deftype CryptoVault [filename keystore password]
  Vault
  (init-vault [vault]
    ... define method body here ...)

  (vault-output-stream [vault]
    ... define method body here ...)

  (vault-input-stream [vault]
    ... define method body here ...)

  IOFactory
  (make-reader [vault]
    (make-reader (vault-input-stream vault)))
  (make-writer [vault]
    (make-writer (vault-output-stream vault))))
```

Notice that the methods for more than one protocol can be defined inline; we have defined the methods for the Vault and IOFactory protocols together, although the bodies of the Vault methods have been elided and will be described next.

The `init-vault` method will generate an Advanced Encryption Standard (AES) key, place it in a `java.security.KeyStore`, write the keystore data to the file specified by the `keystore` field in the `CryptoVault`, and then `password-protect` it.

```
(init-vault [vault]
  (let [password (.toCharArray (.password vault))
        key (.generateKey (KeyGenerator/getInstance "AES"))
        keystore (doto (KeyStore/getInstance "JCEKS")
                      (.load nil password)
                      (.setEntry "vault-key"
                                (KeyStore$SecretKeyEntry. key)
                                (KeyStore$PasswordProtection. password)))]
    (with-open [fos (FileOutputStream. (.keystore vault))]
      (.store keystore fos password))))
```

Both the `vault-output-stream` and `vault-input-stream` methods will use a function, `vault-key`, to load the keystore associated with the `CryptoVault` and extract the AES key used to encrypt and decrypt the contents of the vault.

```
(defn vault-key [vault]
  (let [password (.toCharArray (.password vault))]
    (with-open [fis (FileInputStream. (.keystore vault))]
      (-> (doto (KeyStore/getInstance "JCEKS")
              (.load fis password))
          (.getKey "vault-key" password)))))
```

The `vault-output-stream` method uses the `vault-key` method to initialize an AES cipher object, creates an `OutputStream` from the Vault's filename, and then uses the cipher and `OutputStream` to create an instance of a `CipherOutputStream`.

```
(vault-output-stream [vault]
  (let [cipher (doto (Cipher/getInstance "AES")
                    (.init Cipher/ENCRYPT_MODE (vault-key vault)))]
    (CipherOutputStream. (io/output-stream (.filename vault)) cipher)))
```

The `vault-input-stream` method works just like `vault-output-stream`, except it returns a `CipherInputStream`.

```
(vault-input-stream [vault]
  (let [cipher (doto (Cipher/getInstance "AES")
                    (.init Cipher/DECRYPT_MODE (vault-key vault)))]
    (CipherInputStream. (io/input-stream (.filename vault)) cipher)))
```

To create an instance of a `CryptoVault`, just pass the location where data should be stored, the keystore filename, and the password protecting the keystore.

```
user=> (def vault (->CryptoVault "vault-file" "keystore" "toomanysecrets"))
#'user/vault
```

If the keystore hasn't been initialized yet, call the `init-vault` method:

```
user=> (init-vault vault)
nil
```

Then use the CryptoVault like any other source/destination used by gulp and expectorate.

```
user=> (expectorate vault "This is a test of the CryptoVault")
nil
```

```
user=> (gulp vault)
"This is a test of the CryptoVault"
```

We can use the CryptoVault with the built-in spit and slurp functions by extending it to support the clojure.java.io/IOFactory protocol. This version of the IOFactory has four methods, instead of two like ours, and there are default method implementations defined in a map called default-streams-impl. We'll override just two of its methods, make-input-stream and make-output-stream, by assoc'ing our new implementations into this map and passing it to the extend function.

```
(extend CryptoVault
  clojure.java.io/IOFactory
  (assoc clojure.java.io/default-streams-impl
    :make-input-stream (fn [x opts] (vault-input-stream x))
    :make-output-stream (fn [x opts] (vault-output-stream x))))
```

That's it; now we can read and write to a CryptoVault using slurp and spit.

```
user=> (spit vault "This is a test of the CryptoVault using
spit and slurp")
nil
```

```
user=> (slurp vault)
"This is a test of the CryptoVault using spit and slurp"
```

Let's put all the pieces together in a .clj file. Make a src/examples/datatypes subdirectory within your project directory, and create a file called vault.clj.

```
src/examples/cryptovault_complete.clj
(ns examples.cryptovault-complete
  (:require [clojure.java.io :as io]
            [examples.protocols.io :as proto])
  (:import (java.security KeyStore KeyStore$SecretKeyEntry
                          KeyStore$PasswordProtection)
           (javax.crypto Cipher KeyGenerator CipherOutputStream
                          CipherInputStream)
           (java.io FileInputStream FileOutputStream)))
(defprotocol Vault
  (init-vault [vault])
  (vault-output-stream [vault])
  (vault-input-stream [vault]))
(defn vault-key [vault]
  (let [password (.toCharArray (.password vault))]
    (with-open [fis (FileInputStream. (.keystore vault))]
```

```

    (-> (doto (KeyStore/getInstance "JCEKS")
      (.load fis password))
      (.getKey "vault-key" password))))
(deftype CryptoVault [filename keystore password]
  Vault
  (init-vault [vault]
    (let [password (.toCharArray (.password vault))
          key (.generateKey (KeyGenerator/getInstance "AES"))
          keystore (doto (KeyStore/getInstance "JCEKS")
            (.load nil password)
            (.setEntry "vault-key"
              (KeyStore$SecretKeyEntry. key)
              (KeyStore$PasswordProtection. password)))]
      (with-open [fos (FileOutputStream. (.keystore vault))]
        (.store keystore fos password))))

  (vault-output-stream [vault]
    (let [cipher (doto (Cipher/getInstance "AES")
      (.init Cipher/ENCRYPT_MODE (vault-key vault)))]
      (CipherOutputStream. (io/output-stream (.filename vault)) cipher)))

  (vault-input-stream [vault]
    (let [cipher (doto (Cipher/getInstance "AES")
      (.init Cipher/DECRYPT_MODE (vault-key vault)))]
      (CipherInputStream. (io/input-stream (.filename vault)) cipher)))

  proto/IOFactory
  (make-reader [vault]
    (proto/make-reader (vault-input-stream vault)))
  (make-writer [vault]
    (proto/make-writer (vault-output-stream vault)))

  (extend CryptoVault
    clojure.java.io/IOFactory
    (assoc io/default-streams-impl
      :make-input-stream (fn [x opts] (vault-input-stream x))
      :make-output-stream (fn [x opts] (vault-output-stream x)))))

```

6.5 Records

Classes in object-oriented programs tend to fall into two distinct categories: those that represent programming artifacts, such as `String`, `Socket`, `InputStream`, and `OutputStream`, and those that represent application domain information, such as `Employee` and `PurchaseOrder`.

Unfortunately, using classes to model application domain information hides it behind a class-specific micro-language of setters and getters. You can no longer take a generic approach to information processing, and you end up

with a proliferation of unnecessary specificity and reduced reusability. See Clojure's documentation on datatypes³ for more information.

For this reason, Clojure has always encouraged the use of maps for modeling such information, and that holds true even with datatypes, which is where records come in. A record is a datatype, like those created with `deftype`, that also implements `PersistentMap` and therefore can be used like any other map (mostly); and since records are also proper classes, they support type-based polymorphism through protocols. With records, we have the best of both worlds: maps that can implement protocols.

What could be more natural than using records to play music? So, let's create a record that represents a musical note, with fields for pitch, octave, and duration; then we'll use the JDK's built-in MIDI synthesizer to play sequences of these notes.

Since records are maps, we will be able to change the properties of individual notes using the `assoc` and `update-in` functions, and we can create or transform entire sequences of notes using `map` and `reduce`. This gives us access to the entirety of Clojure's collection API.

We will create a `Note` record with the `defrecord` macro, which behaves just like `deftype`.

```
(defrecord name [& fields] & opts+specs)
```

A `Note` record has three fields: pitch, octave, and duration.

```
(defrecord Note [pitch octave duration])
-> user.Note
```

The pitch will be represented by a keyword like `:C`, `:C#`, and `:Db`, which represent the notes C, C (C sharp), and D (D flat), respectively. Each pitch can be played at different octaves; for instance, middle C is in the fourth octave. Duration indicates the note length; a whole note is represented by 1, a half note by 1/2, a quarter note by 1/4, and a 16th note by 1/16. For example, we can represent a D half note in the fourth octave with this `Note` record:

```
(->Note :D# 4 1/2)
-> #user.Note{:pitch :D#, :octave 4, :duration 1/2}
```

We can treat records like any other datatype, accessing their fields with the dot syntax.

```
(.pitch (->Note :D# 4 1/2))
-> :D#
```

3. <http://clojure.org/datatypes>

But records are also map-like:

```
(map? (->Note :D# 4 1/2))
-> true
```

so we can also access their fields using keywords:

```
(:pitch (->Note :D# 4 1/2))
-> :D#
```

We can create modified records with `assoc` and `update-in`.

```
(assoc (->Note :D# 4 1/2) :pitch :Db :duration 1/4)
-> #user.Note{:pitch :Db, :octave 4, :duration 1/4}

(update-in (->Note :D# 4 1/2) [:octave] inc)
-> #user.Note{:pitch :D#, :octave 5, :duration 1/2}
```

Records are open, so we can associate extra fields into a record:

```
(assoc (->Note :D# 4 1/2) :velocity 100)
-> #user.Note{:pitch :D#, :octave 4, :duration 1/2, :velocity 100}
```

We will use the optional `:velocity` field to represent the force with which a note is played.

When used on a record, both `assoc` and `update-in` return a new record, but the `dissoc` function works a bit differently; it will return a new record if the field being dissociated is optional, like `velocity` in the previous example, but it will return a plain map if the field is mandated by the `defrecord` specification, like `pitch`, `octave`, or `duration`.

In other words, if you remove a required field from a record of a given type, it is no longer a record of that type, and it simply becomes a map.

```
(dissoc (->Note :D# 4 1/2) :octave)
-> {:pitch :D#, :duration 1/2}
```

Notice that `dissoc` returns a map in this case, not a record. One difference between records and maps is that, unlike maps, records are not functions of keywords.

```
((->Note. :D# 4 1/2) :pitch)
-> user.Note cannot be cast to clojure.lang.IFn
```

`ClassCastException` is thrown because records do not implement the `IFn` interface like maps do. This is by design and drives a stylistic difference that makes code more readable.

When accessing a collection, you should place the collection first. When accessing a map that is acting (conceptually) as a data record, you should

place the keyword first, even if the record is implemented as a plain map. Now that we have our basic Note record, let's add some methods so we can play them with the JDK's built-in MIDI synthesizer. We'll start by creating a MidiNote protocol with three methods:

```
src/examples/protocols.clj
```

```
(defprotocol MidiNote
  (to-msec [this tempo])
  (key-number [this])
  (play [this tempo midi-channel]))
```

To play our note with the MIDI synthesizer, we need to translate its pitch and octave into a MIDI key number and its duration into milliseconds. Here we have defined to-msec, key-number, and play, which we will use to create our MidiNote.

- to-msec returns the duration of the note in milliseconds.
- key-number returns the MIDI key number corresponding to this note.
- play plays this note at the given tempo on the given channel.

Now let's extend our Note record to implement the MidiNote protocol.

```
(import 'javax.sound.midi.MidiSystem)
(extend-type Note
  MidiNote
  (to-msec [this tempo]
    (let [duration-to-bpm {1 240, 1/2 120, 1/4 60, 1/8 30, 1/16 15}]
      (* 1000 (/ (duration-to-bpm (:duration this))
                 tempo))))
```

The to-msec function translates the note's duration from whole note, half note, quarter note, and so on, into milliseconds based on the given tempo, which is represented in beats per minute (bpm).

```
(key-number [this]
  (let [scale {:C 0, :C# 1, :Db 1, :D 2,
               :D# 3, :Eb 3, :E 4, :F 5,
               :F# 6, :Gb 6, :G 7, :G# 8,
               :Ab 8, :A 9, :A# 10, :Bb 10,
               :B 11}]
    (+ (* 12 (inc (:octave this)))
       (scale (:pitch this)))))
```

The key-number function maps the keywords used to represent pitch into a number ranging from 0 to 11 [1] and then uses this number along with the given octave to find the corresponding MIDI key-number.⁴

4. Notice more than one pitch maps to 1, 3, 6, 8, and 10.

```
(play [this tempo midi-channel]
  (let [velocity (or (:velocity this) 64)]
    (.noteOn midi-channel (key-number this) velocity)
    (Thread/sleep (to-msec this tempo)))))
```

Finally, the `play` method takes a note, a tempo, and a MIDI channel; sends a `noteOn` message to the channel; and then sleeps for the note's duration. The note continues to play even while the current thread is asleep, stopping only when the next note is sent to the channel.

Now we need a function that sets up the MIDI synthesizer and plays a sequence of notes:

```
(defn perform [notes & {:keys [tempo] :or {tempo 120}}]
  (with-open [synth (doto (MidiSystem/getSynthesizer) .open)]
    (let [channel (aget (.getChannels synth) 0)]
      (doseq [note notes]
        (play note tempo channel)))))
```

The `perform` function takes a sequence of notes and an optional tempo value, opens a MIDI synthesizer, gets a channel from it, and then calls each note's `play` method.

Now that all the pieces are in place, let's make music using a sequence of Note records:

```
(def close-encounters [(->Note :D 3 1/2)
  (->Note :E 3 1/2)
  (->Note :C 3 1/2)
  (->Note :C 2 1/2)
  (->Note :G 2 1/2)])
-> #'user/close-encounters
```

In this case, our “music” consists of the five notes used to greet the alien ships in the movie *Close Encounters of the Third Kind*. To play it, just pass the sequence to the `perform` function:

```
(perform close-encounters)
-> nil
```

We can also generate sequences of notes dynamically with the `for` macro.

```
(def jaws (for [duration [1/2 1/2 1/4 1/4 1/8 1/8 1/8 1/8]
  pitch [:E :F]]
  (Note. pitch 2 duration)))
-> #'user/jaws

(perform jaws)
-> nil
```

The result is the shark theme from *Jaws*—a sequence of alternating E and F notes progressively speeding up as they move from half notes to quarter notes to eighth notes.

Since notes are records and records are map-like, we can manipulate them with any Clojure function that works on maps. For instance, we can map the `update-in` function across the *Close Encounters* sequence to raise, or lower, its octave.

```
(perform (map #(update-in % [:octave] inc) close-encounters))
-> nil
```

```
(perform (map #(update-in % [:octave] dec) close-encounters))
-> nil
```

Or we can create a sequence of notes that have progressively larger values of the optional `:velocity` field:

```
(perform (for [velocity [64 80 90 100 110 120]]
              (assoc (Note. :D 3 1/2) :velocity velocity)))
-> nil
```

This results in a sequence of increasingly more forceful D notes. Manipulating sequences is a particular strength of Clojure, so there are endless possibilities for programmatically creating and manipulating sequences of Note records.

Let's put the `MidiNote` protocol, the Note record, and the `perform` function together in a Clojure source file called `src/examples/midi.clj` so we can use them in the future.

`src/examples/midi.clj`

```
(ns examples.datatypes.midi
  (:import [javax.sound.midi MidiSystem]))
(defprotocol MidiNote
  (to-msec [this tempo])
  (key-number [this])
  (play [this tempo midi-channel]))

(defn perform [notes & {:keys [tempo] :or {tempo 88}}]
  (with-open [synth (doto (MidiSystem/getSynthesizer).open)]
    (let [channel (aget (.getChannels synth) 0)]
      (doseq [note notes]
        (play note tempo channel)))))

(defrecord Note [pitch octave duration]
  MidiNote
  (to-msec [this tempo]
    (let [duration-to-bpm {1 240, 1/2 120, 1/4 60, 1/8 30, 1/16 15}]
      (* 1000 (/ (duration-to-bpm (:duration this))
                 tempo)))))
```

```

(key-number [this]
  (let [scale {:C 0, :C# 1, :Db 1, :D 2,
               :D# 3, :Eb 3, :E 4, :F 5,
               :F# 6, :Gb 6, :G 7, :G# 8,
               :Ab 8, :A 9, :A# 10, :Bb 10,
               :B 11}]
    (+ (* 12 (inc (:octave this)))
      (scale (:pitch this))))
  (play [this tempo midi-channel]
    (let [velocity (or (:velocity this) 64)]
      (.noteOn midi-channel (key-number this) velocity)
      (Thread/sleep (to-msec this tempo)))))

```

6.6 reify

The `reify` macro lets you create an anonymous instance of a datatype that implements either a protocol or an interface. Note that you get access by closure, not by declaration. This is because there are no declared members.

```
(reify & opts+specs)
```

`reify`, like `deftype` and `defrecord`, takes the name of one or more protocols, or interfaces, and a series of method bodies. Unlike `deftype` and `defrecord`, it doesn't take a name or a vector of fields; datatype instances produced with `reify` don't have explicit fields, relying instead on closures.

Let's compose some John Cage-style⁵ aleatoric music⁶ or, better yet, create an aleatoric music generator. We'll use `reify` to create an instance of a `MidiNote` that will play a different random note each time its `play` method is called.

```
src/examples/generator.clj
```

```

(import '[examples.datatypes.midi MidiNote])
(let [min-duration 250
      min-velocity 64
      rand-note (reify
MidiNote
  (to-msec [this tempo] (+ (rand-int 1000) min-duration))
  (key-number [this] (rand-int 100))
  (play [this tempo midi-channel]
    (let [velocity (+ (rand-int 100) min-velocity)]
      (.noteOn midi-channel (key-number this) velocity)
      (Thread/sleep (to-msec this tempo)))))]]
  (perform (repeat 15 rand-note)))

```

5. http://en.wikipedia.org/wiki/John_Cage

6. http://en.wikipedia.org/wiki/Aleatoric_music

The first thing we need to do is import (not use or require) our `MidiNote` protocol from the `examples.midi` namespace. Next we bind two values, `min-duration` and `min-velocity`, that we will use in the `MidiNote` method implementations. Then we use `reify` to create an instance of an anonymous type, which implements the `MidiNote` protocol, that will select a random note, duration, and velocity each time its `play` method is called. Finally, we use the `repeat` function to create a sequence of fifteen notes, consisting of a single instance of `rand-note`, and perform it. *Voilà*, you now have a virtual John Cage!

6.7 Wrapping Up

We covered a lot of ground in this chapter, from the general use of abstraction in programming to some (but not all) of the specific abstraction mechanisms Clojure provides. We explored creating concrete abstractions using protocols in Clojure and had some fun in the process!

But there's still more. Clojure's macro implementation is easy to learn and use correctly for common tasks and yet powerful enough for the harder macro-related tasks. In the next chapter, you will see how Clojure is bringing macros to mainstream programming.